Parallel Algorithms and Algorithmic Analysis Techniques

Algorithm: Parallel Borůvka

Francesco Stabile

20 February 2026

1 Introduction

1.1 Algorithm Description

Borůvka's algorithm[1][2][3] is a greedy algorithm used to determine the *minimum spanning tree* of a connected, weighted, and undirected graph $G(V, E)$. The algorithm proceeds by initializing each vertex of the graph as a distinct connected component. In each iteration, for every component, the minimum-weight edge connecting it to a different component is identified. In case of a tie, a tie-breaking rule is applied to avoid ambiguity and ensure the algorithm remains deterministic. All the selected edges are then added to the solution, progressively merging the connected components and reducing their number at each iteration. The process is repeated until a single connected component remains, which constitutes the desired *minimum spanning tree*. The following images represent some key steps of the sequential algorithm:

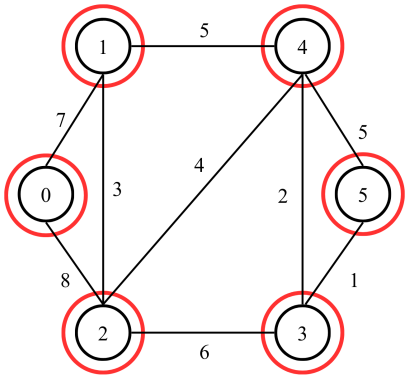


Figure 1: First step of the algorithm: all vertices are distinct components

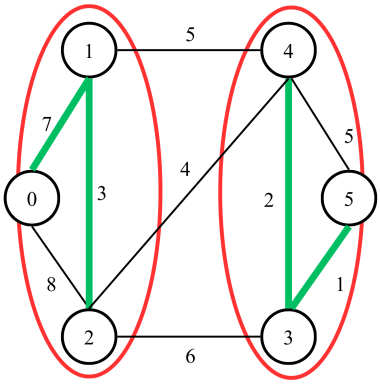


Figure 2: For each component, the minimum-weight edges between distinct components are identified

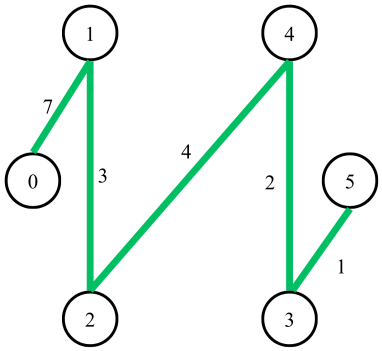


Figure 3: Final step: the only remaining component determines the *minimum spanning tree*

1.2 Pseudocode

Algorithm 1 Borůvka sequenziale

```
1:  MST ← {V, ∅}
2:  components ← G.num_vertices
3:  while components > 1 do
4:      for i ← 1 to components do
5:          safe[i] ← NULL
6:      for all edge uv ∈ E do
7:          if comp(u) ≠ comp(v) then
8:              if safe[comp(u)] = NULL or w(uv) < w(safe[comp(u)]) then
9:                  safe[comp(u)] ← uv
10:             if safe[comp(v)] = NULL or w(uv) < w(safe[comp(v)]) then
11:                 safe[comp(v)] ← uv
12:     for i ← 1 to components do
13:         add safe[i] to MST
14:     components ← CountComponentsAndMap(MST)
15: return MST
```

1.3 Sequential Algorithm Analysis

The `while` loop is executed $O(\log V)$ times, since the number of connected components is reduced by at least half at each iteration. This reduction is guaranteed because each component merges with at least one other component, and in the worst-case scenario, they merge in pairs. The body of the `while` loop takes $O(E)$ time because, during each of its iterations, every one of the E edges of the graph is examined to identify the *safe* edges—meaning those of minimum weight that connect two distinct connected components. Indeed, the remaining operations, such as adding edges to the *minimum spanning tree* or assignment operations, require constant time $\Theta(1)$. In particular, the for loop to determine the *safe* edges takes $O(E)$ iterations, the for loop to reset *safe*[] or to add the *safe*[] edges to *MST* performs $O(V)$ iterations in the worst case, and the CountComponentsAndMap() procedure on *MST* costs $O(V)$, since a *minimum spanning tree* has $V - 1$ edges and V vertices. As the input is assumed to be a connected graph, we have $V \leq E + 1$, thus $O(V + E) = O(E)$, meaning the total complexity is equal to $O(E \log V)$.

2 Parallel Design

2.1 Algorithm Selection: Borůvka - Prim - Kruskal

For the *minimum spanning tree* problem, the primary choices are the algorithms of Borůvka, Prim[1], and Kruskal[1]. The selection of Borůvka is justified by its enormous potential for parallelism during the first iterations, whereas Prim or Kruskal offer fewer possibilities because a large part of their execution remains sequential. In particular, the crucial factor driving the choice toward Borůvka is the ability to select edges independently for each component, thereby enabling workload balancing among processes, an aspect less exploitable in the other two alternatives. The parallel potential found in Prim's algorithm is restricted only to weight updates, as selecting the next vertex to include depends on previous choices. Finally, in Kruskal's algorithm, it would be possible to parallelize the sorting of edge weights, but not the remaining phases. For these reasons, Borůvka was chosen as the ideal candidate with the highest parallel potential compared to the other two alternatives, although it still presents significant challenges in the final iterations when the number of connected components is dramatically reduced.

2.2 Proposed Strategy

Several strategies and optimizations were tested before achieving acceptable scalability. The first challenge to address was choosing how to distribute the graph among the various processors, and thus how to partition the *input*. Although initially partitioning the V vertices seemed like the optimal choice, since it allowed obtaining the global minimum edge independently, with no communication required in the first iterations—a more thorough analysis led to the preference of distributing the edges instead. Each processor is therefore responsible for a subset of edges.
The main advantage provided by edge distribution is a more balanced workload; indeed, by distributing vertices along with their incident edges, some processes could end up with significantly more edges to process, thereby increasing the *overhead* caused by *idling*. This partitioning strategy was maintained across all proposed approaches and is illustrated below.

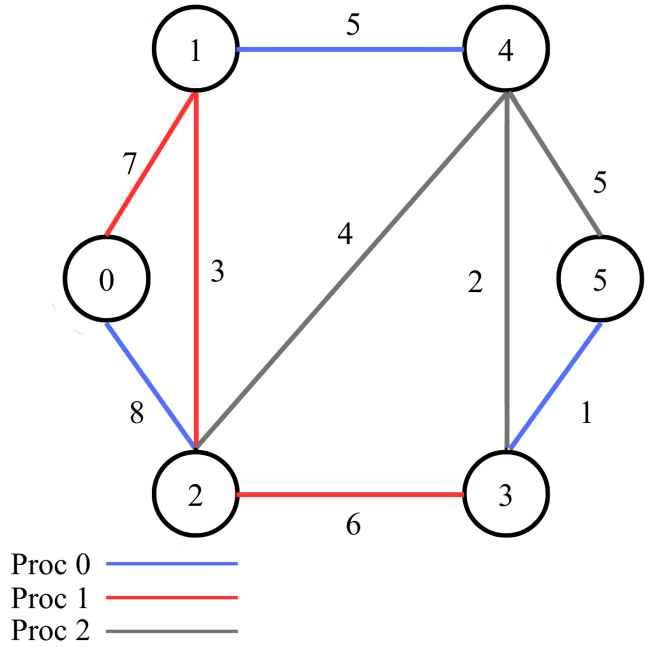


Figure 4: Example of edge *Scatter* with 3 processors

To keep track of the connected components during the algorithm's iterations, since each processor must know which component every vertex belongs to, it was chosen to use a *Union-Find* data structure[4][5] replicated on each processor. For the *find* and *union* functions, *path compression* and *union by rank* are used respectively to optimize complexity and achieve $O(\alpha(V))$, where $\alpha(V)$ is the inverse Ackermann function and can be treated as a constant, which justifies the choice of this structure. The implementation used for the *Union-Find* structure is shown below:

```
typedef struct UF {
    igraph_int_t rank;
    igraph_int_t parent;
} UnionFind;

//Initialize the structure
void init_union_find(UnionFind* uf, igraph_int_t
   n_vertex, igraph_int_t* num_components) {
    for (int i = 0; i < n_vertex; i++) {
        uf[i].parent = i;
        uf[i].rank = 0;
```

```
    }
    *num_components = n_vertex;
}

//Find the component to which vertex belongs by
   traversing the structure and compressing the paths
igraph_int_t find(UnionFind* uf, igraph_int_t vertex) {
    if(uf[vertex].parent == vertex){
        return vertex;
    }
    uf[vertex].parent = find(uf, uf[vertex].parent);
    return uf[vertex].parent;
}

//Merge components containing v1 and v2 and return 1 if
   a merge occurred, 0 otherwise.
//Also update num_components to the new value after the
   potential merge.
int merge_components(UnionFind* uf, igraph_int_t v1,
   igraph_int_t v2, igraph_int_t* num_components) {
    int r1 = find(uf, v1);
    int r2 = find(uf, v2);
    if(r1 != r2){
        if(uf[r1].rank < uf[r2].rank){
            uf[r1].parent = r2;
        }
        else if(uf[r1].rank > uf[r2].rank){
            uf[r2].parent = r1;
        }
        else{
            uf[r2].parent = r1;
            uf[r1].rank += 1;
        }
        *num_components -= 1;
        return 1;
    }
    return 0;
}
```

1. Inefficient AllReduce

The first strategy involved searching for the local minimum edges for each component seen by each process. Subsequently, an *AllReduce* collective com-

munication with a *custom* operator allows each process to obtain the global minimum edge per component. Finally, the various processes apply the *merges* in the same order to keep the *Union-Find* coherent and synchronized.

```
//Custom operator for the AllReduce operation
//Selects the minimum edge based on weight and insert it
   into the output array
    void find_min_edge(void* edge_in, void* edge_inout,
       int* len, MPI_Datatype* dptr){
    Edge* e_in = (Edge*) edge_in;
    Edge* e_out = (Edge*) edge_inout;
    for (int i = 0; i < *len; i++) {
        if (e_in[i].weight < e_out[i].weight) {
            e_out[i] = e_in[i];
        }
    }
}
```

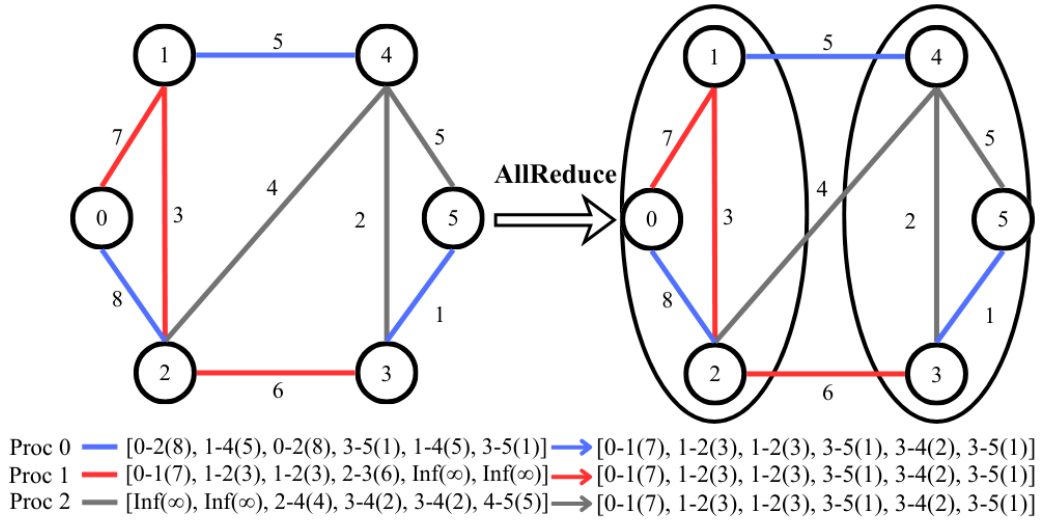


Figure 5: Example of *AllReduce* with 3 processors

In this first attempt, however, the data volume exchanged with the *AllReduce* does not scale with the components, which in the worst-case scenario are

halved. In the subsequent approach, we attempted to optimize this aspect, achieving decent results.

2. Resizing AllReduce

The core optimization of this strategy is to reduce the weight of each performed *AllReduce*; indeed, once the merges between components are executed, the latter are significantly reduced, and subsequent collective communications will have an increasingly lower impact if executed on the number of remaining active components. To achieve this, it was necessary to introduce a *mapping* phase so that all processes would index the component roots in the exact same way.

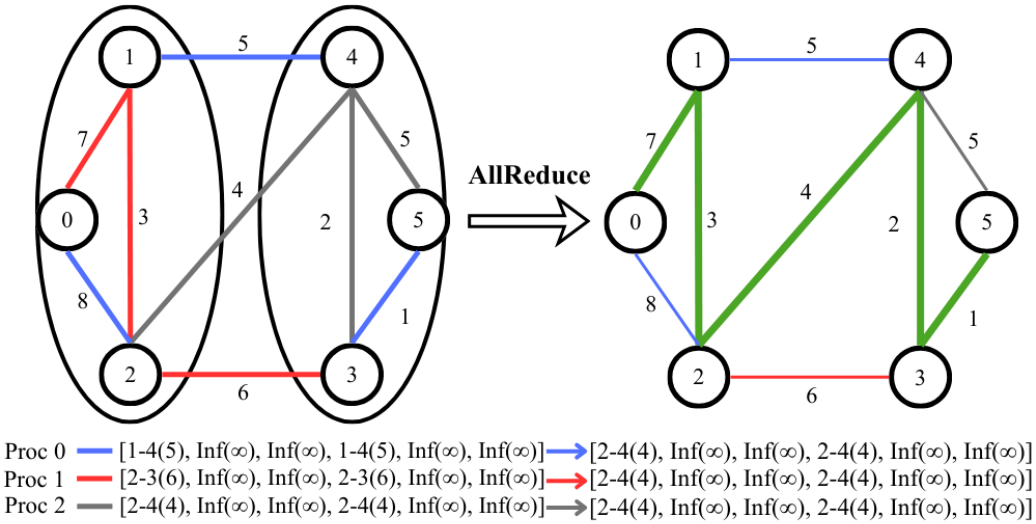


Figure 6: *AllReduce* performed in the first strategy

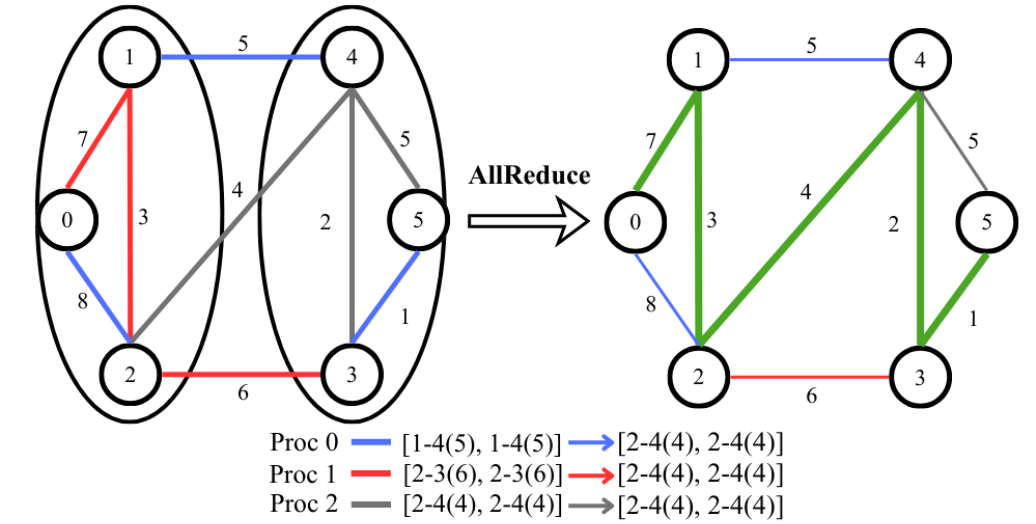


Figure 7: Optimized *AllReduce* in the second approach

With this adjustment, the communication overhead is substantially reduced, but performance is still not satisfactory; in fact, a *speedup* of 1.7 is

achieved, with a relative efficiency of approximately 42%. The search phase for local minimum edges can be further optimized, since even after the first iteration, many edges belonging to each processor become internal to the components; therefore, checking them again at every iteration increases execution time. This idea led to the final version, which is detailed below.

3. Final Version

The proposed idea is identical to the previous ones, but includes optimizations that make it possible to achieve good performance. Indeed, the search for local minimum edges is reduced at each iteration to the number of currently active edges. An edge is considered active as long as it connects two vertices belonging to two distinct components; by doing so, internal edges are no longer redundantly processed. To achieve this, an edge *cleanup* phase is required after the merges. This phase requires no communication and can be carried out in parallel. To further speed up the search phase of the subsequent iteration, the incident vertices of each process's local edges can be overwritten with the new component roots formed by the merges. In this way, there is no need to look up the component of a vertex, and the search is reduced to a simple memory access.

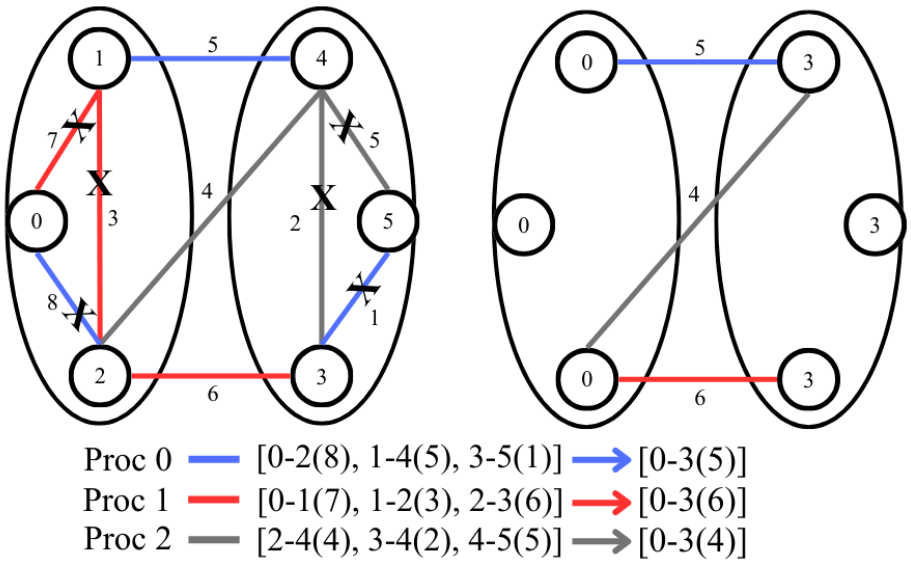


Figure 8: Example of *cleanup* with 3 processors

Finally, the minimum-search function used for the *AllReduce* operation

was modified, as it lacked an explicit tie-breaking rule for edges with equivalent minimum weights; consequently, in some scenarios, the correctness of the algorithm was compromised. The new implementation is shown below.

```
// New implementation with tie-breaking rule
// If the weight is the same, the edge with the minimum
   source vertex is chosen
// If the source vertex is the same, the edge with the
   minimum destination vertex is chosen
void find_min_edge(void* edge_in, void* edge_inout, int*
   len, MPI_Datatype* dptr) {
    Edge* e_in = (Edge*) edge_in;
    Edge* e_out = (Edge*) edge_inout;
    for (int i = 0; i < *len; i++) {
        if (e_in[i].weight < e_out[i].weight) {
            e_out[i] = e_in[i];
        }
        else if (e_in[i].weight == e_out[i].weight &&
           e_in[i].weight < IGRAPH_INFINITY) {
            if (e_in[i].from < e_out[i].from) {
                e_out[i] = e_in[i];
            } else if (e_in[i].from == e_out[i].from &&
               e_in[i].to < e_out[i].to) {
                e_out[i] = e_in[i];
            }
        }
    }
}
```

3 Analysis of the Proposed Solution

Within this section, the computational and space complexities are analyzed, as well as the isoefficiency relation and the scalability function.

3.1 Computational Complexity

The complexity analysis can be divided into four sections inside the `while` loop and an additional part outside the main loop. Calculating the variables for edge distribution takes $\Theta(p)$, while initializing the structures necessary for the computation, such as `initUnionFind`, costs $O(V)$. Inside the loop, which costs $O(\log V)$ iterations, we find four phases listed below:

Mapping: All V vertices are scanned to find their belonging component. Once obtained, the indexing is re-performed from 0 to the number of active components -1; thus, it takes $O(V)$.

Search: Searching within the assigned edges to obtain the local minima takes $O(\frac{E}{p})$.

Comm: This includes the collective communication analyzed below and the merging of the components. The latter takes $O(V)$ merges in the worst case, and each of them costs $O(\alpha(V))$, where $\alpha(V)$ is the inverse Ackermann function. Therefore, it can be concluded that the total cost is $O(V)$.

Cleanup: The local edges that have already been merged are removed; those internal to the components are searched for within the active edges and removed. Therefore, the total cost is $O(\frac{E}{p})$.

Multiplying the cost of a single iteration by the $O(\log V)$ iterations, we get $O(\log V(\frac{E}{p} + V)) = O(\frac{E}{p} \log V + V \log V)$.

3.2 Space Complexity

Each processor must maintain the *UnionFind* and support arrays for *mapping* and communication in memory, in addition to the assigned edges. Therefore, the final complexity is $O(V + \frac{E}{p})$ per processor.

3.3 Communication Complexity

The communication cost, excluding I/O, is dominated by the single collective operation `AllReduce`. Since it is invoked on the number of active components, the number of elements exchanged decreases at each iteration. In particular, we know that in the worst case, the components are halved; therefore, the complexity is given by:

$$O(\sum_{i=0}^{\log V} \frac{V}{2^i} \log p) = O(V \log p \sum_{i=0}^{\log V} (\frac{1}{2})^i) = O(V \log p)$$

3.4 Isoefficiency Function

Sequential time: $O(E \log V)$.

Parallel time: $O(\frac{E}{p} \log V + V \log V)$.

Communication complexity: $O(V \log p)$.

Parallel overhead: $O(p(\frac{E}{p} \log V + V \log V + V \log p) - E \log V) = O(pV \log V + Vp \log p)$.

Isoefficiency relation: Since the parallel overhead involves multiple terms, we need to find the isoefficiency relation for each of them and consider the asymptotically larger term. In particular, we consider the case of dense graphs where $O(E) = O(V^2)$:

1. $E \log V \geq CpV \log V \Rightarrow V \geq Cp$ ideal scalability.
2. $E \log V \geq CVp \log p \Rightarrow V \log V \geq Cp \log p$ which is the same condition as the first term.

For sparse graphs, where $O(E) = O(V)$, it is easy to see that scalability is not achieved. This is due to the fact that the work performed by each processor on the edges becomes dominated by communication—since all V vertices are communicated to synchronize the *UnionFind*—and by the work of *mapping* and merging components. In fact, as the number of processors increases, $\frac{E}{p}$ becomes progressively negligible compared to V when $O(E) = O(V)$. It is concluded that excellent scalability is achieved only for dense graphs.

Scalability function: $M(V, E) = O(V + E)$. Considering dense graphs, we have $M(V, E) = O(V^2)$; therefore, $\frac{M(f(p))}{p} = \frac{C^2p^2}{p} = C^2p$, which yields acceptable scalability only for certain values of p.
In the case of sparse graphs, scalability is not achieved.

3.5 Benchmark and Speedup

For the analysis of the implementation, graphs of different sizes have been generated using the following method[6]:

```
// n = number of vertices
igraph_real_t p = 2 * log(n) / n;
error = igraph_erdos_renyi_game_gnp(&graph, n, p,
   IGRAPH_UNDIRECTED, IGRAPH_SIMPLE_SW,
   IGRAPH_EDGE_UNLABELED);
```

The value of p makes it very likely that the final graph is connected, otherwise an error is returned, therefore all generated graphs have the guarantee of being connected. To verify the correctness, the sequential Prim algorithm

was used. The results are obtained on an average of one hundred iterations for each scenario.

	V = 1000 E = 6753	V = 10000 E = 92201	V = 100000 E = 1150788
p = 1	0.000248s	0.003505s	0.051609s
p = 2	0.000229s V,E should be increased	0.002662s V,E should be increased	0.024768s Ideal Speedup
p = 3	0.000320s CO dominates	0.002456s V,E should be increased	0.021331s Sublinear Speedup
p = 4	0.000460s CO dominates	0.002699s V,E should be increased	0.018317s Sublinear Speedup

Table 1: Execution time analysis for different orders of V and E on Erdos Renyi graphs.

Analyzing them, it is evident that communication dominates for a few thousand edges and vertices, but a significant benefit is achieved when increasing by an order of magnitude. Averaging one hundred iterations on a graph with 200,000 vertices and 2,441,945 edges, it takes ∼0.117s on a single processor, whereas it takes ∼0.0377s on four processors; therefore, a speedup of ∼3.11 and a relative efficiency of 78% are achieved. Furthermore, the proposed implementation turns out to be highly sensitive to the *input*. The previous results were obtained from executions on randomly generated graphs; these exhibit a high correlation among edges since the latter are ordered by creation within the file. Consequently, it benefits from greater data locality, as edges that are very close in the graph also tend to appear close within the file. This makes execution faster. To simulate a random ordering of the edges and analyze performance in the absence of this correlation provided by the graph generator, a shuffling algorithm[7] was used on the *input* file, which only modifies the order of the edges while leaving the graph unchanged. Subsequently, the *benchmarks* were repeated.

	V = 1000 E = 6753	V = 10000 E = 92201	V = 100000 E = 1150788
p = 1	0.000298s	0.004511s	0.075610s
p = 2	0.000275s V,E should be increased	0.002706s Sublinear Speedup	0.033716s Superlinear Speedup
p = 3	0.000307s CO dominates	0.002937s V,E should be increased	0.023810s Superlinear Speedup
p = 4	0.000416s CO dominates	0.002951s V,E should be increased	0.020585s Linear Speedup

Table 2: Execution time analysis for different orders of V and E on Erdos Renyi graphs with edge shuffling.

The results show that while performance in terms of absolute execution time worsens, the algorithm remains fairly robust. The degradation occurs mostly on the single processor, whereas the parallel execution times, although also worsened, remain similar to the previous scenario. In this specific context, the benefit provided by parallel computing is greater compared to the previous scenario, since the sequential work is much more sensitive to the *input*; consequently, sequential performance degrades, while parallel performance remains fairly stable.

3.6 Amdahl's and Gustafson's Metrics

By dividing the two scenarios into *Ordered* and *Shuffled* to indicate, respectively, the case where the input features a graph with a strong correlation among edges and the case where the input lacks a specific ordering, we obtain the following theoretical *Speedup* values using Gustafson's and Amdahl's laws.

Ordered: Using 4 processors, 0.002507s are spent executing sequential code out of a total of 0.018317s; consequently, 13.69% is spent on sequential code. Therefore, the scaled *speedup* according to Gustafson's law is as follows:

$$Scaled\ Speedup \leq s + (1 - s)p = 0.1369 + (1 - 0.1369)4 = 3.59$$

On a single processor, the percentage is equal to 4.86% (0.002507s out of 0.051609s); therefore, the *speedup* predicted by Amdahl's law is:

$$Speedup \leq \frac{1}{f + \frac{1-f}{p}} = \frac{1}{0.0486 + \frac{1-0.0486}{4}} = 3.49$$

Shuffled: Using 4 processors, 0.002521s are spent executing sequential code out of a total of 0.020585s; consequently, 12.25% is spent on sequential code. Therefore, the scaled *speedup* according to Gustafson's law is as follows:

$$Scaled\ Speedup \leq s + (1 - s)p = 0.1225 + (1 - 0.1225)4 = 3.63$$

On a single processor, the percentage is equal to 3.33% ($0.002521s$ out of $0.075610s$); therefore, the *speedup* predicted by Amdahl's law is:

$$Speedup \leq \frac{1}{f + \frac{1-f}{p}} = \frac{1}{0.0333 + \frac{1-0.0333}{4}} = 3.63$$

3.7 PC Specifications

All *benchmarks* were performed on a MacBook Air M2 2022 with 4 Performance Cores.

References

[1] University of Illinois, *CS 374 AL1 - Algorithms for Minimum Spanning Trees*, `https://courses.grainger.illinois.edu/cs374al1/fa2024/lec/slides/21_mst.pdf`

[2] GeeksforGeeks, *Borůvka's Algorithm - Greedy Algo-9*, `https://www.geeksforgeeks.org/dsa/boruvkas-algorithm-greedy-algo-9/`

[3] Wikipedia, *Borůvka's algorithm*, `https://en.wikipedia.org/wiki/Bor%C5%AFvka%27s_algorithm`

[4] GeeksforGeeks, *Introduction to Disjoint Set Data Structure (Union-Find Algorithm)*, `https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/`

[5] CP-Algorithms, *Disjoint Set Union*, `https://cp-algorithms.com/data_structures/disjoint_set_union.html`

[6] Mathematics Stack Exchange, *Connectivity of a graph in the Erdős–Rényi model*, `https://math.stackexchange.com/questions/4087765/connectivity-of-a-graph-in-the-erd%C5%91s-r%C3%A9nyi-model`

[7] Wikipedia, *Fisher–Yates shuffle*, `https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle`